Project 1

Vincenzo Illiano
(Dated: September 10, 2026)

https://github.uio.no/vincenzi/FYS3150-Project1.git

INTRODUCTION:

The main topic of this project is the numerical solution of the one-dimensional Poisson equation:

$$-\frac{d^2u}{dx^2} = f(x), \tag{1}$$

where $f(x)$ is a known function and $u(x)$ is the unknown function. In the following text, we will assume that $f(x) = 100e^{-10x}$ and that $x \in [0, 1]$. We will use the boundary condition:

$$u(0) = u(1) = 0 \tag{2}$$

This provides enough information to find a unique solution for eq. 1 analytically.

PROBLEM 1

We can show that $u(x) = 1 - (1 - e^{-10})x - e^{-10x}$ is a solution for our problem by simply showing that eq. 1 and eq. 2 are satisfied. We can see that: $u'(x) = -1 + e^{-10} + 10e^{-10x} \implies u''(x) = -100e^{-10x} = -f(x)$. This shows that eq. 1 is satistfied. Eq. 2 is also satisfied, in fact: $u(0) = 1 - (1 - e^{-10}) \cdot 0 - e^{-10 \cdot 0} = 1 - 1 = u(1) = 1 - (1 - e^{-10}) \cdot 1 - e^{-10 \cdot 1} = 1 - 1 = 0$.

PROBLEM 2

We can plot the function $u(x)$ by calculating the points $(x, u(x))$ in the c++ program `problem2.cpp`. In particoular, the function `double u(double x)` returns the value $u(x)$ if $0 \leq x \leq 1$, 0 otherwise. In the `main` function the program then stores `N` values of x into a vector and the `N` corresponding values of $u(x)$ into another vector. These are then saved into a `.txt` file in two colums. For now, only 4 decimal places are stored in the final output. For all the problems after problem 2, 16 digits will be stored. The Python program `plotfile.py` then reads the output file of `problem2.cpp` and plots the two colums using `matplotlib`. With `N` = 100, the program produces the plot in fig. 1.

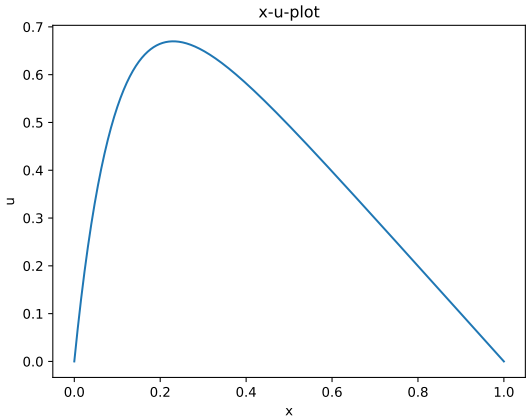


FIG. 1: Plot of $u(x)$ with `N` = 100

PROBLEM 3

We can use the fact that for any continuous and differentiable function $u(x)$, given any real number h

$$\frac{d^2u}{dx^2}\Big|_{x=x_i} = u_i'' = \frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + O(h^2), \tag{3}$$

where $u_i = u(x_i)$ and $u_{i\pm1} = u(x_i \pm h)$. We can therefore write eq. 1 as:

$$\frac{u_{i+1} - 2u_i + u_{i-1}}{h^2} + O(h^2) = -f_i,$$

where $f_i = f(x_i)$. This equation becomes easily solvable if we omit the $o(h^2)$. By doing this, however, we are not going to find the actual solution $u(x)$, but an approximation of the solution, which we'll call $v(x)$. In particoular we expect that $v \to u$ as $h \to 0$. We have now *discretized* Poisson's equation, which we can rewrite as:

$$-v_{i-1} + 2v_i - v_{i+1} = h^2 f_i \tag{4}$$

PROBLEM 4

We can now turn the discretized Poisson's equation into a matrix equation $A\mathbf{v} = \mathbf{g}$. In order to do so, let

$$\mathbf{g} = \begin{pmatrix} h^2 f_1 + v_0 \\ h^2 f_2 \\ ... \\ h^2 f_{n-2} \\ h^2 f_{n-1} + v_n \end{pmatrix}, \ \mathbf{v} = \begin{pmatrix} v_1 \\ v_2 \\ ... \\ v_{n-2} \\ v_{n-1} \end{pmatrix}.$$

Note that we are assuming that the original equation is defined over the interval $x \in [x_0, x_n]$ and we are assuming that v_0 and v_n are known boundary conditions. Let A be the $n-1 \times n-1$ tridiagonal matrix

$$A = \begin{pmatrix} 2 & -1 & 0 & 0 & ... & 0 \\ -1 & 2 & -1 & 0 & ... & 0 \\ 0 & ... & ... & ... & ... & 0 \\ 0 & ... & 0 & -1 & 2 & -1 \\ 0 & ... & 0 & 0 & -1 & 2 \end{pmatrix}.$$

Let's observe that $(A\mathbf{v})_i = -v_{i-1} + 2v_i - v_{i+1}$, except for $i = 1$ and $i = n-1$, where we get $2v_1 - v_2$ and $-v_{n-2} + 2v_{n-1}$ respectively. Since we defined g_1 and g_{n-1} with an additional $+v_0$ and $+v_n$ respectively, this is not a problem. In conclusion, using the $\mathbf{g}$ and the A we just defined, eq. 4 simply becomes $A\mathbf{v} = \mathbf{g}$.

PROBLEM 5

Let $\mathbf{v}^*$ be a complete solution, in particoular:

$$\mathbf{v}^* = \begin{pmatrix} v_0 \\ v_1 \\ ... \\ v_{n-1} \\ v_n \end{pmatrix}$$

When we solve for $\mathbf{v}$ with the method described in *problem 4*, we only solve for $v_1, v_2, ...v_{n-1}$ using an $n-1 \times n-1$ matrix [1]. The values v_0 and v_n are priorly known from the boundary conditions.

PROBLEM 6

A

We will use the Gaussian elimination algorithm for the special case of a tridiagonal matrix, also known as the *thomas algorithm*. It's known that given a tridiagonal matrix

$$A = \begin{pmatrix} b_1 & c_1 & 0 & ... & 0 \\ a_2 & b_2 & c_2 & ... & 0 \\ 0 & a_3 & b_3 & ... & 0 \\ ... & ... & ... & ... & ... \\ 0 & ... & a_n & b_n & c_n \end{pmatrix},$$

then the equation $A\mathbf{v} = \mathbf{g}$ is solved by:

$$\begin{cases} v_n = \frac{\tilde{g}_n}{\tilde{b}_n} \\ v_i = \frac{\tilde{g}_i - c_i v_{i+1}}{\tilde{b}_i} \quad i = 1, 2, ..., n-1 \end{cases}, \tag{5}$$

where

$$\begin{cases} \tilde{b_1} = b_1 \\ \tilde{b}_i = b_i - \frac{a_i}{\tilde{b}_{i-1}} c_{i-1} \quad i = 2, 3, ..., n \\ \tilde{g}_1 = g_1 \\ \tilde{g}_i = g_i - \frac{a_i}{\tilde{b}_{i-1}} \tilde{g}_{i-1} \quad i = 2, 3, ..., n \end{cases}. \tag{6}$$

The idea of Thomas'algorithm therefore is to compute $\tilde{b_i}$ and $\tilde{g}_i$ starting from $i = 1$ (*forward substitution*) and then compute $\tilde{v}_i$ starting from $i = n$ (*backward substitution*). This can easily be achieved by two for cycles. Note that since in C++ all vectors start from the index 0, the indices in the code may differ from those in eq. 5 and 6. This does not change the substance of the algorithm. The algorithm has been implemented in the function `thomas_algorithm` in the file `project1functions.cpp`.

B

We can see from eq. 6 that in the forward substitution we are performing 2 subtractions, 2 multiplications and 2 divisions $n-1$ times, where n is the size of vector $\mathbf{b}$. From eq. 6 we can see that in the backward substitution we are performing 1 subtraction, 1 multiplication and 1 division for $n-1$ times, plus 1 other division for v_n. This adds up to a total of FLOPs $= 6 \cdot (n-1) + 3 \cdot (n-1) + 1 = 9n - 8$.

PROBLEM 7

A

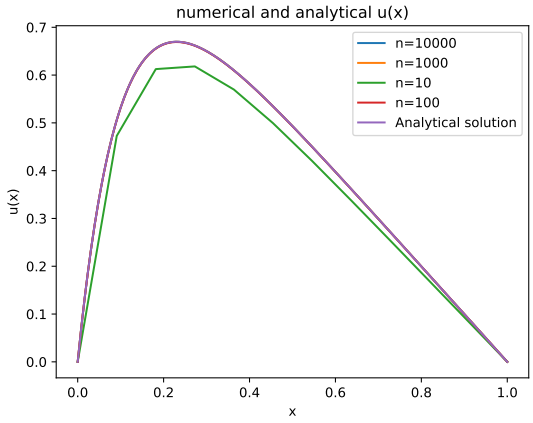
The problem is solved with the `solvePoisson` function in `project1functions.cpp`. This function:

1. Initializes the $\mathbf{x}$, $\mathbf{g}$, $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ vectors, without including the boundary values $x = 0$ and $x = 1$;

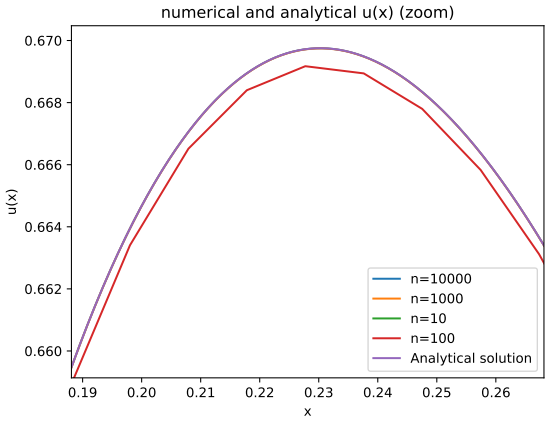
2. calls the `thomas_algorithm` function in order to find $\mathbf{v}$;

3. includes the boundary elements $(x = 0, v = 0)$ and $(x = 1, v = 0)$ to the $\mathbf{x}$ and $\mathbf{v}$ vectors;

4. writes these two to an output file



((a)) Numerical vs analytical solution



((b)) Numerical vs analytical solution zoomed in

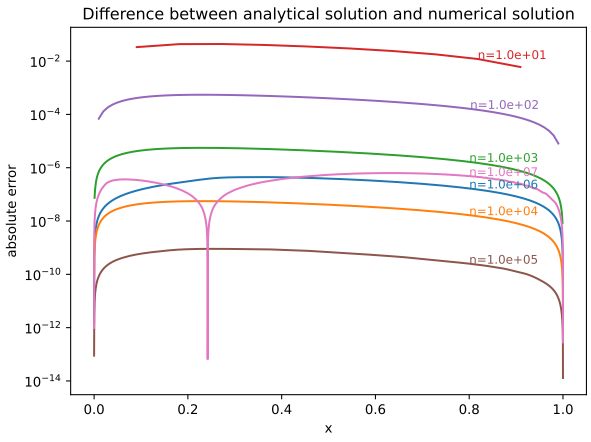
FIG. 2

B

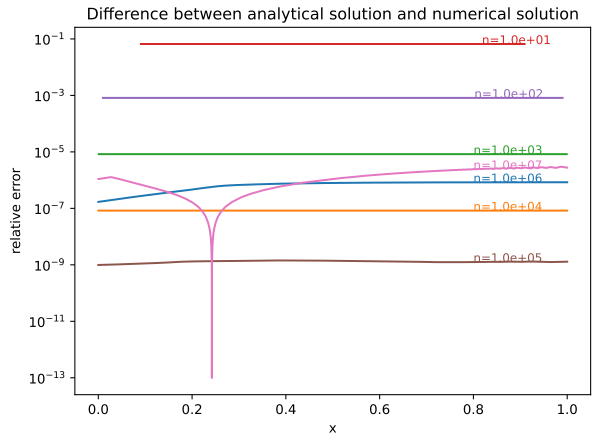
This in done in `problem7.cpp` and in `plotNumericalSolutions.py`. Note that each numerical solution is stored in a different file, named something like `n[number].txt`, where `[number]` is the value of n for that file. In `plotNumericalSolutions.py` all files are found and stored into a list, so that they can then be cycled through one by one. The analytical solution has been calculated for $N = 100000$ points. As can be seen in fig. 2, the differences between the numerical and the analytical solutions are barely visible at this scale unless $n < 100$. It is interesting to see that the numerical solution tends to underestimate the analytical one, as can be easily seen for $n = 10$.

PROBLEM 8

This is done in the Python script `plotErrors.py`. Like `plotNumericalSolutions.py`, the script finds all the numerical solutions that have been calculated and stores their filenames into a list.



((a)) Absolute error of numerical solutions



((b)) Relative error of numerical solutions

FIG. 3

In fig. 3(a) and 3(b) it's possible to see the absolute error of the numerical solutions compared to the analytical one and the relative error respectively, in log scale.

n	10^1	10^2	10^3	10^4	10^5	10^6	10^7
$\max(\lvert\epsilon\rvert)$	$6.612 \cdot 10^{-2}$	$8.165 \cdot 10^{-4}$	$8.317 \cdot 10^{-6}$	$8.331 \cdot 10^{-8}$	$1.436 \cdot 10^{-9}$	$8.404 \cdot 10^{-7}$	$2.984 \cdot 10^{-6}$

TABLE I: Maximum relative error of numerical solutions for each n

In table I and in fig. 4 it's possible to see the maximum relative error for each n in log scale. It's interesting to see how the logarithm of the relative error ϵ seems to linearly decrease as the logarithm of n increases, until it reaches a minimum and starts increasing again. Before *epsilon* reaches its minimum, the main source of error is probably the truncation error due to v being only an approximation of u. This error is expected to go to 0 as n goes to infinity. However, as n increases, roundoff error becomes non negligible, and ϵ starts to increase again.

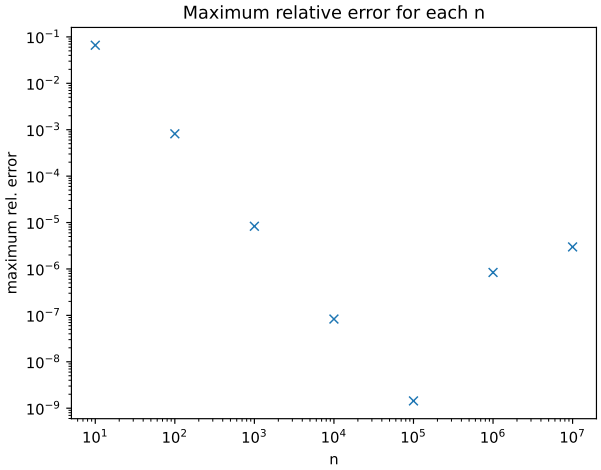


FIG. 4: Maximum relative error of the numerical solutions for each n

PROBLEM 9

We can structure the special algorithm exactly like the algorithm implemented in the function `thomas_algorithm` for problem 6, with the difference that every `a[i]`, `b[i]` and `c[i]` entries will be changed respectively with a -1, a 2 and a -1. This has been implemented in the `specialAlgorithm` function in `project1functions.cpp`. Similarly to `thomas_algorithm`, this algorithm has two for cycles where almost all the operations take place. In the first for cycle 1 subtraction, 1 sum and 2 divisions are executed $n-1$ times, for a total of $4(n-1)$ FLOPs. In the second for cycle 1 sum and 1 division is executed for $n-1$ times, for a total of $2(n-1)$ FLOPs. Counting one other FLOP outside the two cycles, the algorithm has $2(n-1)+4(n-1)+1 = 6n-5$ FLOPs. This is less compared to the general algorithm, which had $9n-8$ FLOPs.

PROBLEM 10

This problem is solved in the file `problem10.cpp`. After preparing the $\mathbf{x}, \mathbf{v}, \mathbf{a}, \mathbf{b}$ and $\mathbf{c}$ vectors, both the general and the special algorithm are executed for $N = 10^4$ times, and their average execution time is measured and saved into a file for values of n ranging from 10^1 to 10^6. In fig. 5 it's possible to see the average timings in log scale. As expected, the timings for the general algorithm are always greater than the ones for the special algorithm. It's interesting to plot this data in a linear scale, as shown in fig. 6. For ease of viewing, straight lines have been drawn between the points. We can see clearly that both the time for the general and the special algorithm grows linearly with n, but

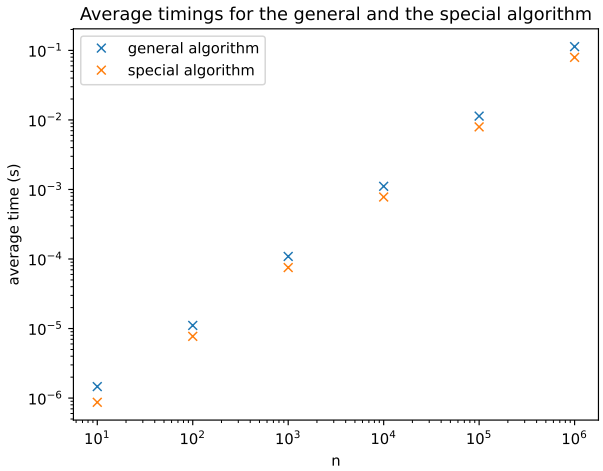


FIG. 5: Average timings for the special and the general algorithm in log scale

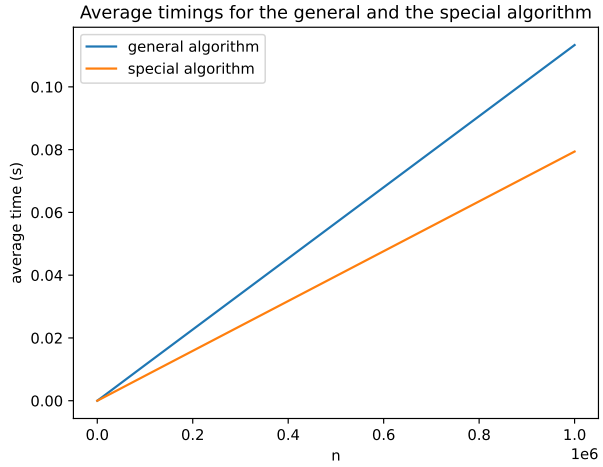


FIG. 6: Average timings for the special and the general algorithm in linear scale

with a bigger slope for the general algorithm, as expected from the number of FLOPs we calculated.

---

[1] I did not answer the questions of problem 5 directly to have continuity with what I had written in problem 4 (I also used an n instead of an m). The exact and direct answer would be: (a): $n = m - 2$, (b): We find all the v_i exept for the boundary ones.